Tienda de ropa alternativa

Ficha inicial de proyecto personal para registro académico

Modalidad individual | Infraestructura de Servicios | 19 de octubre de 2026

Problema y propuesta

La tienda ofrece ropa alternativa, como playeras y prendas de estilo gótico, punk y urbano. Necesita controlar existencias por talla y gestionar compras de piezas de edición limitada sin vender unidades que ya están comprometidas. Propongo conservar el catálogo, el carrito y las ventas del sistema actual, y añadir reservas temporales de inventario y seguimiento del pedido para que el cliente conozca el avance de su compra.

Usuarios y permisos

Cliente: consulta prendas y tallas, agrega productos al carrito, genera pedidos y consulta o cancela sus compras cuando el estado lo permite. Administrador: gestiona productos, tallas, existencias y usuarios, confirma pagos simulados, prepara pedidos y registra entregas. La autorización se validará en el servidor; cada cliente solo podrá consultar y modificar sus propios pedidos.

Alcance funcional

Se conservarán autenticación, catálogo de ropa, tallas, carrito, pedidos, usuarios y reporte de ventas. Se incorporarán el control de ediciones limitadas, reservas temporales por talla e historial de pedidos. Habrá búsqueda, paginación y reporte de ventas y unidades disponibles por talla. Productos y usuarios tendrán baja lógica; los pedidos se cancelarán sin borrar su historial. La semilla reproducible incluirá al menos 100 registros de negocio relacionados y cuentas ficticias.

Quedan fuera la fabricación de prendas, personalización, pagos bancarios reales, facturación fiscal e integración con paqueterías. El pago y la entrega se registrarán de forma simulada. El ambiente productivo será un laboratorio académico con datos ficticios.

Reglas de negocio propias

1. Reserva por talla. Agregar una prenda al carrito no reservará inventario. Al generar el pedido se reservarán sus unidades durante 30 minutos, solo si hay disponibilidad en cada talla. La operación será transaccional para impedir que dos clientes aparten la misma unidad.

2. Límite en ediciones limitadas. Cada cliente podrá comprar como máximo dos unidades de un mismo modelo marcado como edición limitada. El límite sumará todas sus tallas y pedidos vigentes o entregados; no contará pedidos cancelados o vencidos. El servidor comprobará el límite aunque se creen varios pedidos.

3. Vencimiento y pago. Solo el administrador confirmará un pago simulado mientras la reserva siga vigente. Si transcurren los 30 minutos sin confirmación, el pedido vencerá y liberará sus unidades una sola vez. La confirmación y el vencimiento se coordinarán para evitar dobles movimientos de inventario.

4. Cancelación y conservación de la compra. El cliente podrá cancelar únicamente pedidos pendientes de pago, liberando su reserva. Al confirmar el pedido se conservarán el precio unitario, la talla y la descripción comprada, aunque después cambie el catálogo. Solo se marcarán entregados pedidos previamente preparados.

Diferenciación del proyecto

La propuesta mantiene el negocio de ropa alternativa y añade un proceso verificable para vender ediciones limitadas: límite acumulado por cliente, reservas con vencimiento e historial de movimientos por talla. Estas reglas afectan compras e inventario y pueden probarse con pedidos concurrentes. La diferenciación se apoya en su funcionamiento; el docente validará el registro para evitar duplicados.

Flujo y diseño técnico propuesto

Flujo con cambios de estado

Carrito → Pendiente de pago → Confirmado → En preparación → Listo para entrega → Entregado.

El carrito precede al pedido. Al finalizar la compra se validan tallas y límites, se fijan los precios y se crea el pedido pendiente con su reserva. El administrador confirma el pago simulado y avanza la preparación y la entrega. Desde pendiente también podrá pasar a Cancelado por solicitud del cliente o a Vencido por tiempo agotado. Ambas salidas liberan la reserva. La API rechazará saltos de estado y registrará actor, fecha y motivo.

Módulos y adaptación del sistema actual

Se reutilizarán los módulos PHP de autenticación, usuarios, productos, tallas, carrito, pedidos y ventas, conservando la estética de ropa alternativa. Se ampliarán inventario con reservas y vencimientos, productos con la marca de edición limitada y pedidos con estados e historial consultable. Las nuevas funciones se desarrollarán sobre el sistema existente; esta ficha describe el alcance previsto.

Tecnologías y servicios

Interfaz web con HTML, CSS y JavaScript, servida por Nginx y comunicada por HTTPS con una API PHP. El backend conservará la estructura modular de CodeIgniter y se actualizará desde la versión 3.0.4 encontrada, verificando compatibilidad con una versión mantenida de PHP. Se separará la interfaz de las vistas actuales para desplegar frontend y backend de forma independiente. Las sesiones se compartirán en PostgreSQL para permitir varias réplicas sin depender del disco local.

Docker empaquetará los servicios; Kubernetes administrará frontend, API, proceso de sincronización y ambos motores de datos. Se crearán dos clústeres independientes: desarrollo con al menos un nodo y producción de laboratorio con un control plane y dos workers. Producción tendrá al menos dos réplicas de frontend y backend y HPA de 2 a 6 en el backend. Cada ambiente tendrá HTTPS, secretos y volúmenes propios; se añadirán políticas de red, sondas, respaldos, métricas y alertas.

Uso de PostgreSQL como base relacional

Será la fuente de verdad de usuarios, roles, prendas, tallas, inventario, pedidos, detalles, precios y reservas. Las entidades con CRUD serán productos, usuarios y pedidos: cada pedido pertenece a un usuario y sus detalles referencian productos y tallas. Se usarán claves, restricciones e índices. Las transacciones protegerán el límite de compra, las reservas, el pago simulado y la liberación de existencias.

Uso y valor de MongoDB como base no relacional

Almacenará una bitácora consultable de pedidos e inventario: reserva creada, pago confirmado, cambio de estado, cancelación y vencimiento. Cada tipo de evento tendrá detalles distintos, como unidades liberadas por talla o motivo de cancelación. Los documentos permitirán reunir esos datos variables sin añadir columnas por cada tipo de evento. El cliente verá la línea de tiempo de su compra y el administrador podrá investigar movimientos de stock.

Cada evento tendrá identificador único, pedido, tipo, fecha, actor y datos del movimiento. Se validará su estructura según el tipo, con índices por pedido y fecha y por producto y talla. Este historial será una función visible de la aplicación y permitirá explicar por qué cambió la disponibilidad de una prenda.

Consistencia y recuperación entre motores

Cada cambio y su evento pendiente se guardarán en una misma transacción de PostgreSQL mediante una tabla outbox. Un proceso enviará los eventos a MongoDB con escritura idempotente por identificador único. Si MongoDB falla, la compra conservará su estado y el historial indicará sincronización pendiente. El proceso reintentará sin duplicados. Los límites, reservas y transiciones siempre se validarán contra PostgreSQL; se probará la recuperación al restablecer MongoDB.